Architecture

Group Number: **Cohort 1, Group 11**
Group Name: **Y111 Studios**

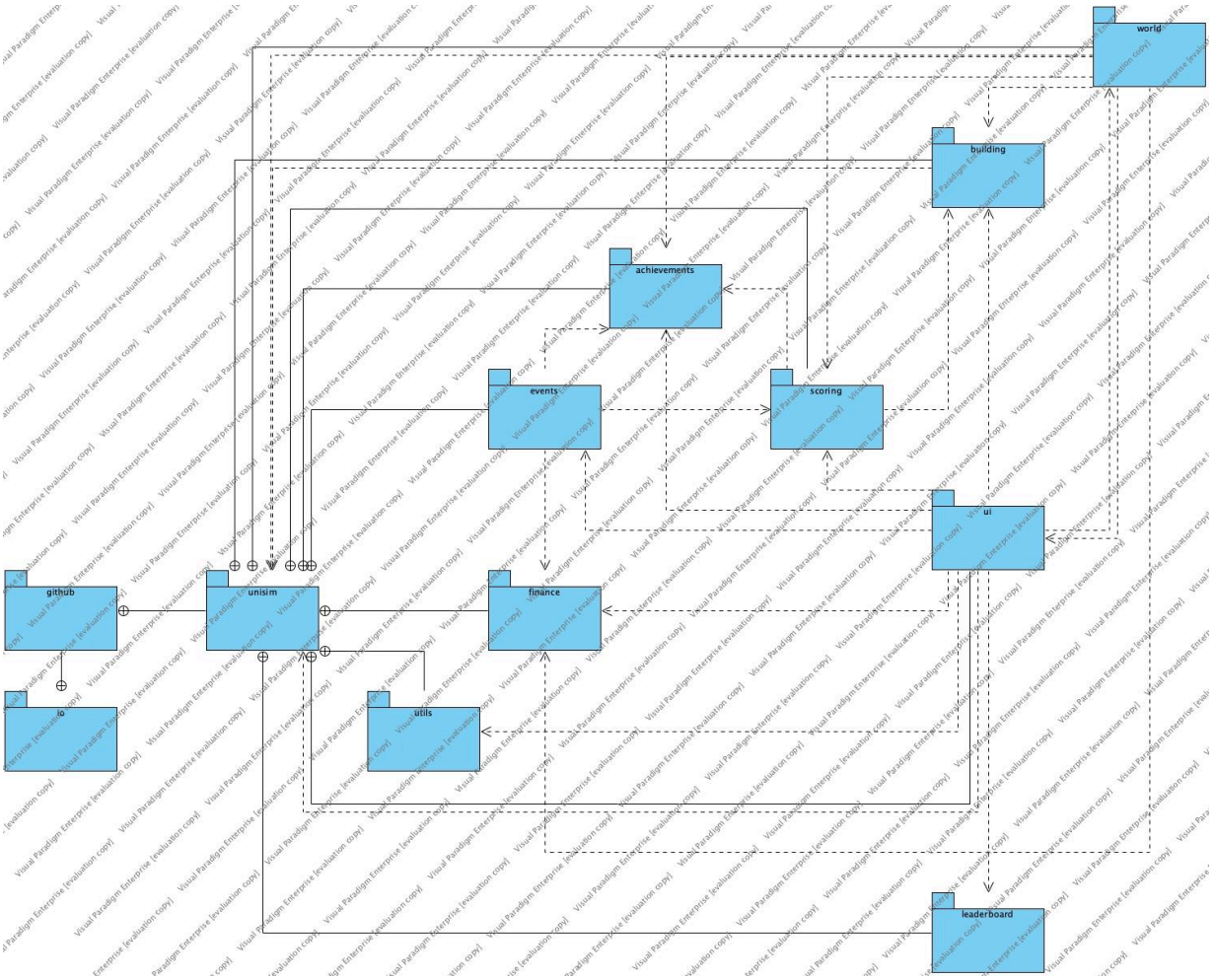
Group Members:

1. James Hutchinson
2. Somto Igweonu
3. Robert Kisloski
4. Sam Knight
5. Kenneth Kok
6. Ashish Kumar

This document outlines the architecture phase of the game development process, including design decisions based on user and system requirements. UML class, package, state, and sequence diagrams were used to represent the game's structure and behaviour.

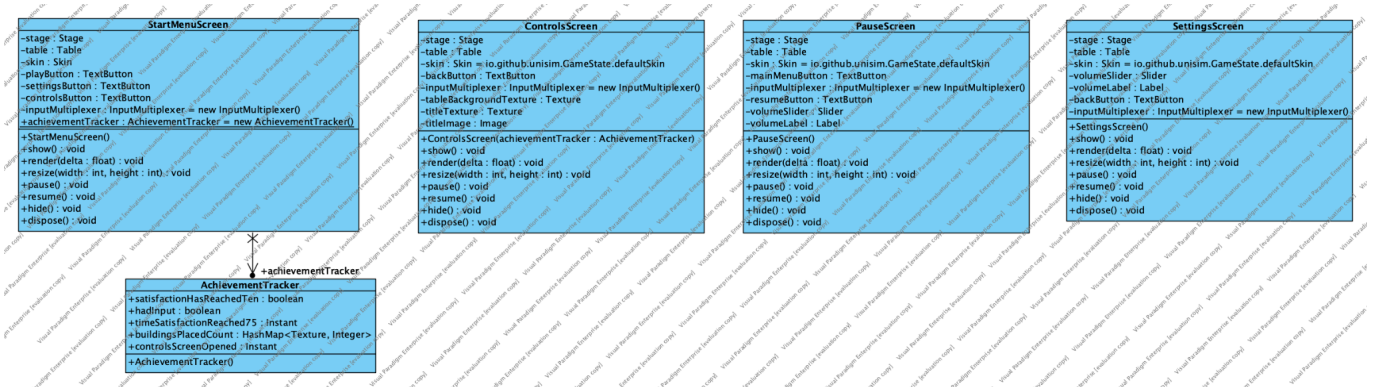
PlantUML, Graphviz, PlantUML Web Server, Draw.io, and Visual Paradigm were used to create the diagrams.

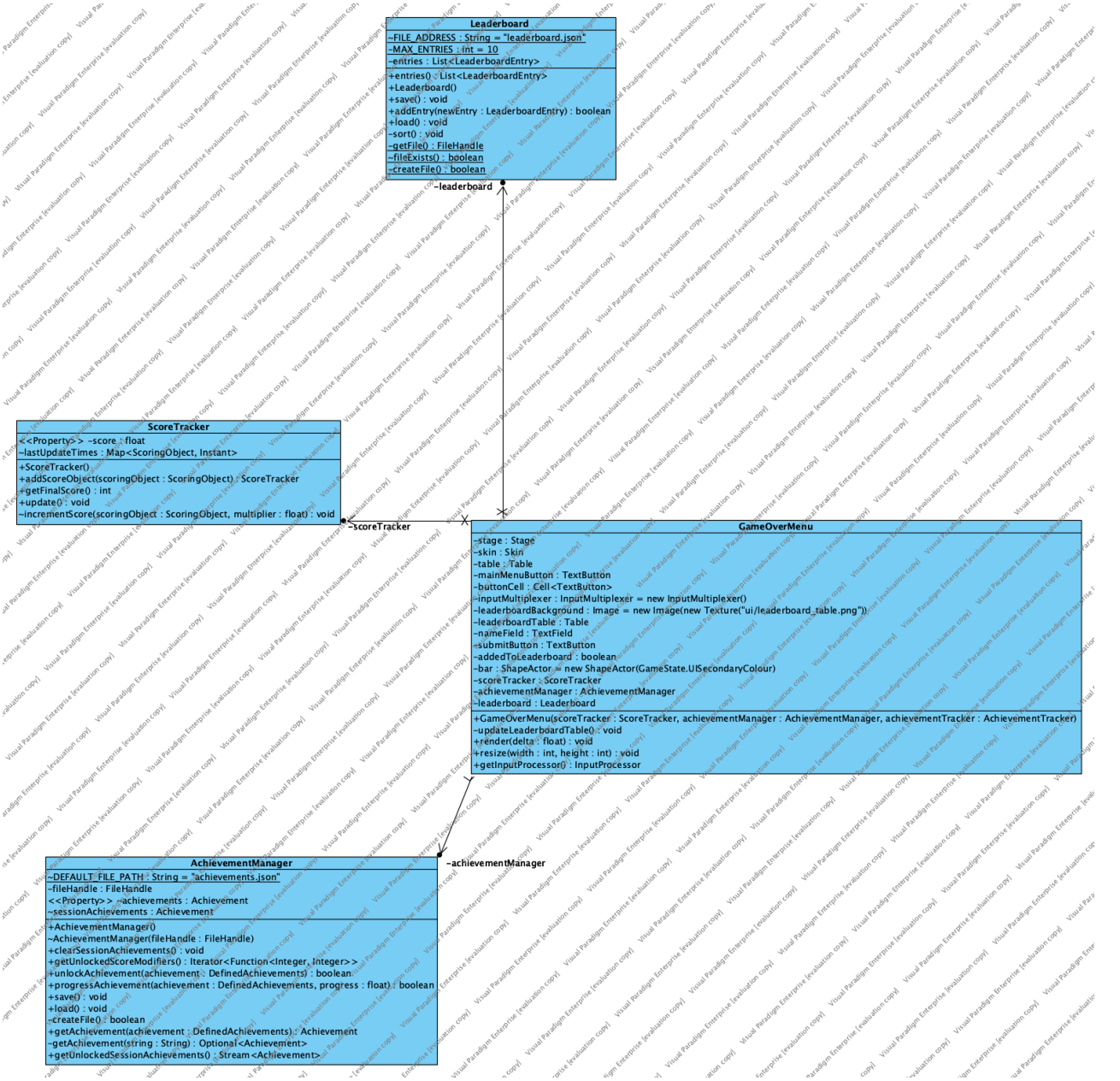
Package Diagram

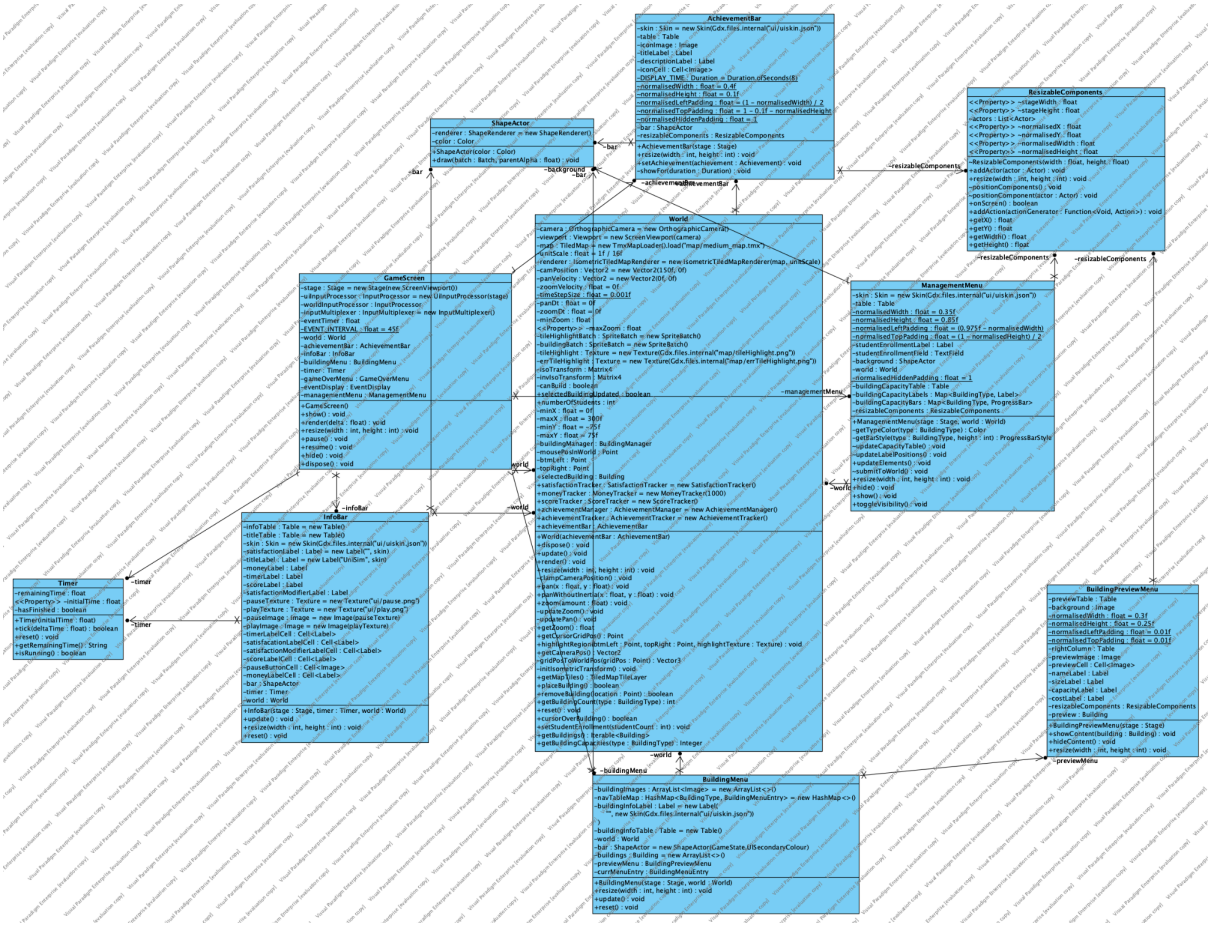


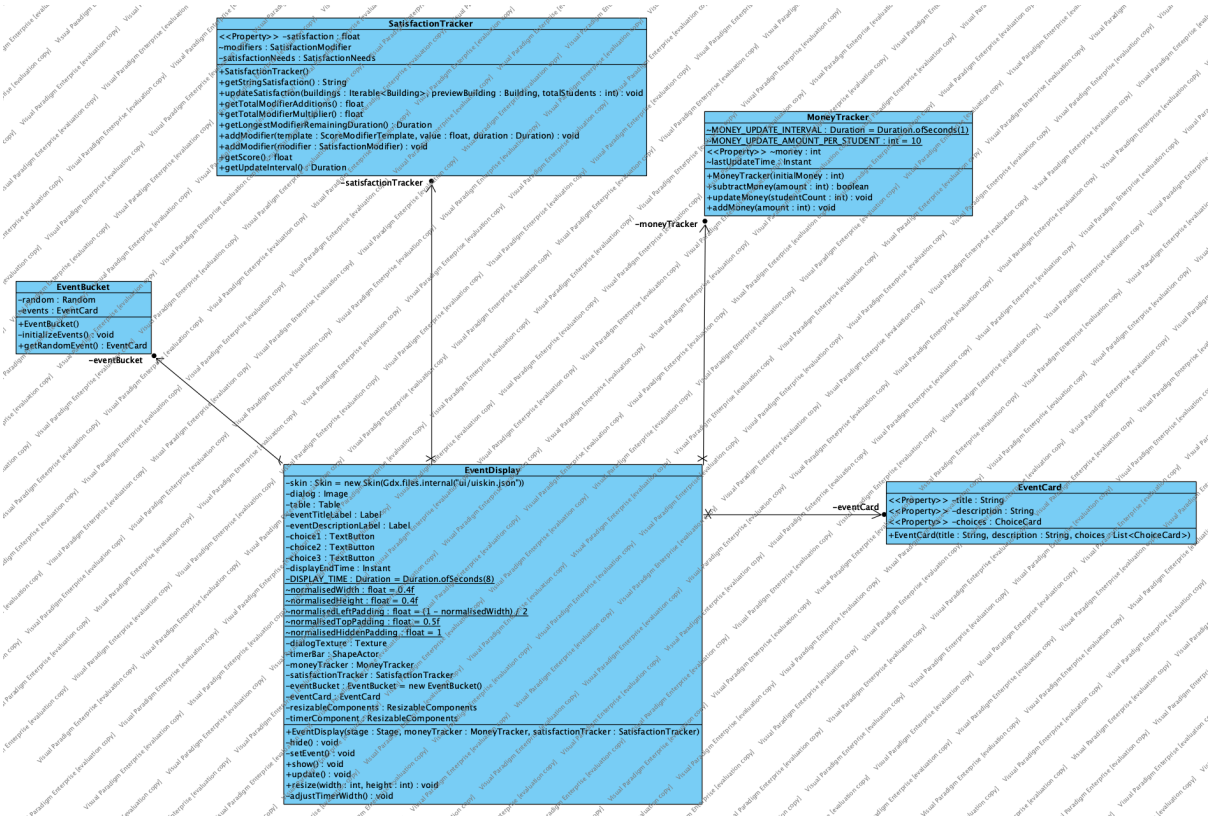
Structure Diagrams for Each Package

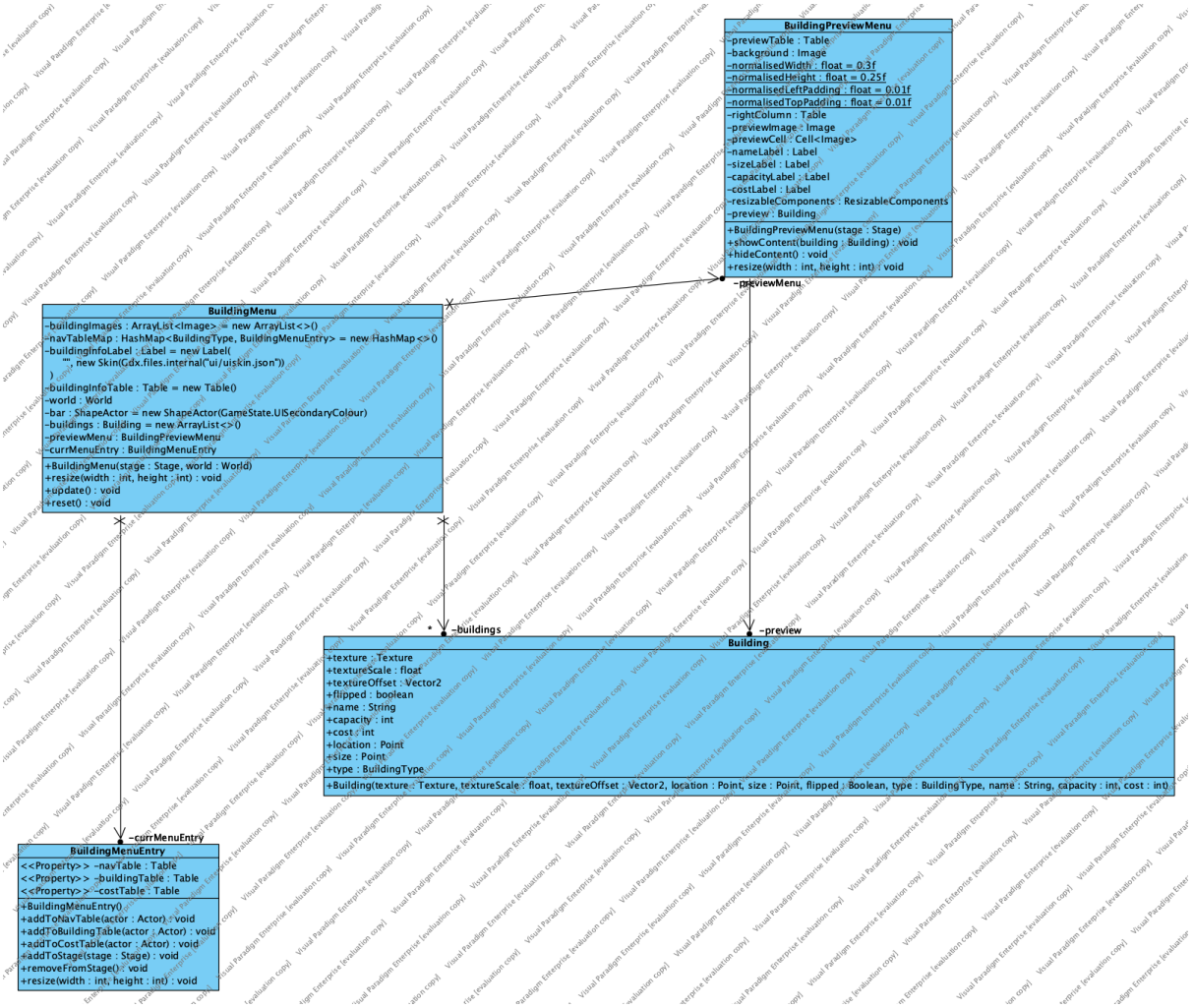
UI: The UI diagrams were split up as the full diagram was too big for one page. The UI package contains a lot of classes but most of these are just different states that allow for features like a pause menu and settings screen to be added.

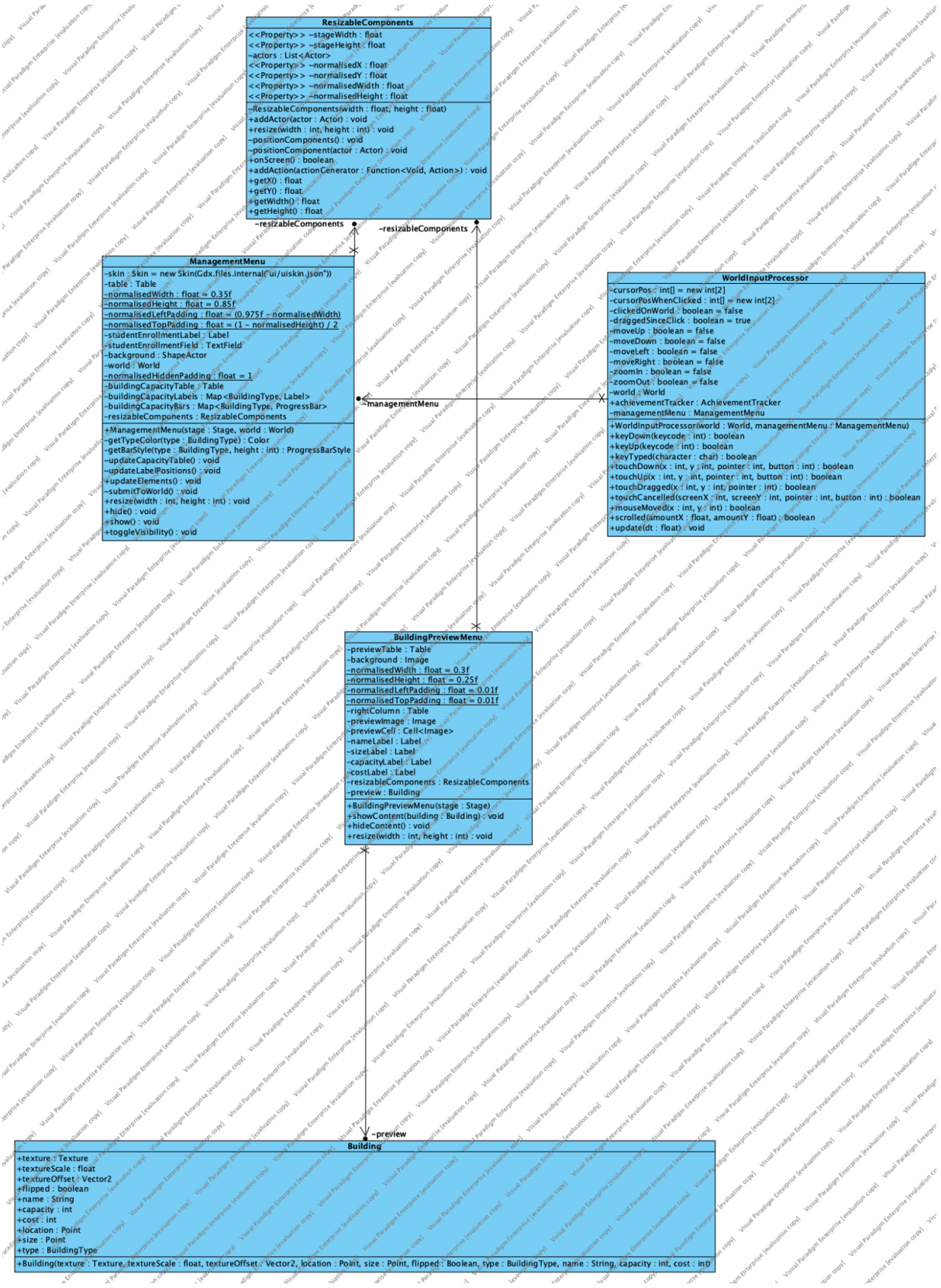

StartMenuScreen
-stage : Stage
-table : Table
-skin : Skin
-playButton : TextButton
-settingsButton : TextButton
-controlsButton : TextButton
-inputMultiplexer : InputMultiplexer = new InputMultiplexer()
+achievementTracker : AchievementTracker = new AchievementTracker()
+StartMenuScreen()
+show() : void
+render(delta : float) : void
+resize(width : int, height : int) : void
+pause() : void
+resume() : void
+hide() : void
+dispose() : void
+achievementTracker
AchievementTracker
+satisfactionHasReachedTen : boolean
+hadInput : boolean
+timeSatisfactionReached75 : Instant
+buildingsPlacedCount : HashMap<Texture, Integer>
+controlsScreenOpened : Instant
+AchievementTracker()
ControlsScreen
-stage : Stage
-table : Table
-skin : Skin = io.github.unisim.GameState.defaultSkin
-backButton : TextButton
-inputMultiplexer : InputMultiplexer = new InputMultiplexer()
-tableBackgroundTexture : Texture
-titleTexture : Texture
-titleImage : Image
+ControlsScreen(achievementTracker : AchievementTracker)
+show() : void
+render(delta : float) : void
+resize(width : int, height : int) : void
+pause() : void
+resume() : void
+hide() : void
+dispose() : void
PauseScreen
-stage : Stage
-table : Table
-skin : Skin = io.github.unisim.GameState.defaultSkin
-mainMenuButton : TextButton
-inputMultiplexer : InputMultiplexer = new InputMultiplexer()
-resumeButton : TextButton
-volumeSlider : Slider
-volumeLabel : Label
+PauseScreen()
+show() : void
+render(delta : float) : void
+resize(width : int, height : int) : void
+pause() : void
+resume() : void
+hide() : void
+dispose() : void
SettingsScreen
-stage : Stage
-table : Table
-skin : Skin = io.github.unisim.GameState.defaultSkin
-volumeSlider : Slider
-volumeLabel : Label
-backButton : TextButton
-inputMultiplexer : InputMultiplexer = new InputMultiplexer()
+SettingsScreen()
+show() : void
+render(delta : float) : void
+resize(width : int, height : int) : void
+pause() : void
+resume() : void
+hide() : void
+dispose() : void


Leaderboard
~FILE_ADDRESS : String = "leaderboard.json"
~MAX_ENTRIES : int = 10
~entries : List<LeaderboardEntry>
+entries() : List<LeaderboardEntry>
+Leaderboard()
+save() : void
+addEntry(newEntry : LeaderboardEntry) : boolean
+load() : void
~sort() : void
~getFile() : FileHandle
~fileExists() : boolean
~createFile() : boolean
~leaderboard
ScoreTracker
<<Property>> ~score : float
~lastUpdateTimes : Map<ScoringObject, Instant>
+ScoreTracker()
+addScoreObject(scoringObject : ScoringObject) : ScoreTracker
+getFinalScore() : int
+update() : void
~incrementScore(scoringObject : ScoringObject, multiplier : float) : void
~scoreTracker
GameOverMenu
-stage : Stage
-skin : Skin
-table : Table
-mainMenuButton : TextButton
-buttonCell : Cell<TextButton>
-inputMultiplexer : InputMultiplexer = new InputMultiplexer()
-leaderboardBackground : Image = new Image(new Texture("ui/leaderboard_table.png"))
-leaderboardTable : Table
-nameField : TextField
-submitButton : TextButton
-addedToLeaderboard : boolean
-bar : ShapeActor = new ShapeActor(GameState.UISecondaryColour)
-scoreTracker : ScoreTracker
-achievementManager : AchievementManager
~leaderboard : Leaderboard
+GameOverMenu(scoreTracker : ScoreTracker, achievementManager : AchievementManager, achievementTracker : AchievementTracker)
~updateLeaderboardTable() : void
+render(delta : float) : void
+resize(width : int, height : int) : void
+getInputProcessor() : InputProcessor
~achievementManager
AchievementManager
~DEFAULT_FILE_PATH : String = "achievements.json"
~fileHandle : FileHandle
<<Property>> ~achievements : Achievement
~sessionAchievements : Achievement
+AchievementManager()
~AchievementManager(fileHandle : FileHandle)
+clearSessionAchievements() : void
+getUnlockedScoreModifiers() : Iterator<Function<Integer, Integer>>
+unlockAchievement(achievement : DefinedAchievements) : boolean
+progressAchievement(achievement : DefinedAchievements, progress : float) : boolean
+save() : void
+load() : void
~createFile() : boolean
+getAchievement(achievement : DefinedAchievements) : Achievement
~getAchievement(string : String) : Optional<Achievement>
+getUnlockedSessionAchievements() : Stream<Achievement>

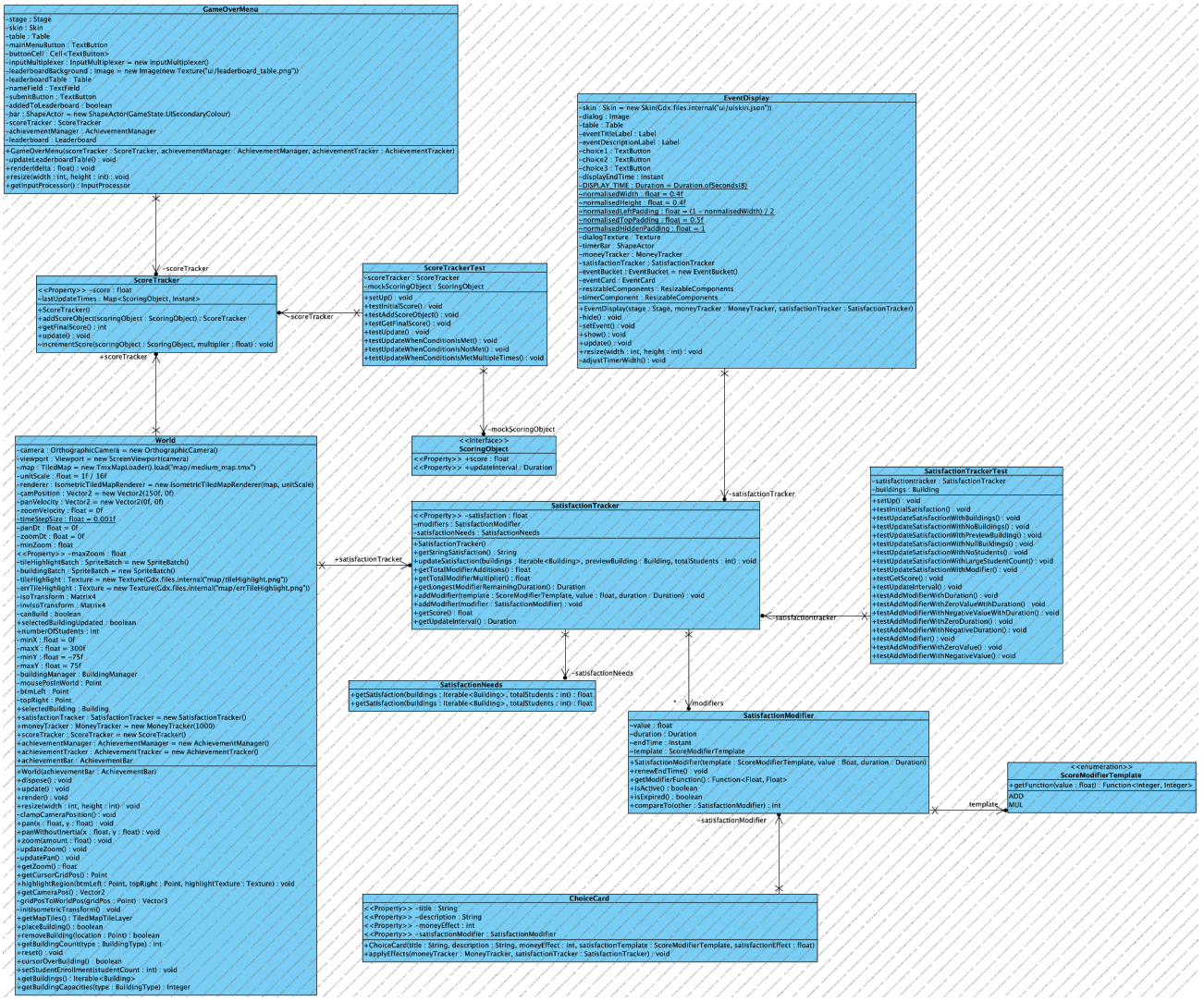

AchievementBar
-skin : Skin = new SkinCdx.files.internal("ui/uiskin.json")
-table : Table
-iconImage : Image
-titleLabel : Label
-descriptionLabel : Label
-iconCell : Cell<Image>
-DISPLAY_TIME : Duration = Duration.ofSeconds(8)
-normalisedWidth : float = 0.4f
-normalisedHeight : float = 0.1f
-normalisedLeftPadding : float = (1 - normalisedWidth) / 2
-normalisedTopPadding : float = 1 - 0.1f - normalisedHeight
-normalisedHiddenPadding : float = 1
-bar : ShapeActor
-resizableComponents : ResizableComponents
+AchievementBar(stage : Stage)
+resize(width : int, height : int) : void
+setAchievement(achievement : Achievement) : void
+showFor(duration : Duration) : void
ShapeActor
-renderer : ShapeRenderer = new ShapeRenderer()
-color : Color
+ShapeActor(color : Color)
+draw(batch : Batch, parentAlpha : float) : void
-background
-bar
-bar
-bar
-bar
ResizableComponents
<<Property>> ~stageWidth : float
<<Property>> ~stageHeight : float
-actors : List<Actor>
<<Property>> ~normalisedX : float
<<Property>> ~normalisedY : float
<<Property>> ~normalisedWidth : float
<<Property>> ~normalisedHeight : float
~ResizableComponents(width : float, height : float)
+addActor(actor : Actor) : void
+resize(width : int, height : int) : void
-positionComponents() : void
-positionComponent(actor : Actor) : void
+onScreen() : boolean
+addActionsCreationGenerator : Function<Void, Action>) : void
+getX() : float
+getY() : float
+getWidth() : float
+getHeight() : float
-resizableComponents
-resizableComponents
-resizableComponents
-achievementBar
GameScreen
-stage : Stage = new Stage(new ScreenViewport())
-uiInputProcessor : InputProcessor = new UInputProcessor(stage)
-worldInputProcessor : InputProcessor
-inputMultiplexer : InputMultiplexer = new InputMultiplexer()
-eventTimer : float
-EVENT_INTERVAL : float = 45f
-world : World
-achievementBar : AchievementBar
-infoBar : InfoBar
-buildingMenu : BuildingMenu
-timer : Timer
-gameOverMenu : GameOverMenu
-eventDisplay : EventDisplay
-managementMenu : ManagementMenu
+GameScreen()
+show() : void
+render(delta : float) : void
+resize(width : int, height : int) : void
+pause() : void
+resume() : void
+hide() : void
+dispose() : void
World
-camera : OrthographicCamera = new OrthographicCamera()
-viewport : Viewport = new ScreenViewport(camera)
-map : TiledMap = new TmxMapLoader().load("map/medium_map.tmx")
-unitScale : float = 1f / 16f
-renderer : IsometricTiledMapRenderer = new IsometricTiledMapRenderer(map, unitScale)
-camPosition : Vector2 = new Vector2(150f, 0f)
-partyVelocity : Vector2 = new Vector2(0f, 0f)
-zoomVelocity : float = 0f
-timeStepSize : float = 0.001f
-panDt : float = 0f
-zoomDt : float = 0f
-minZoom : float
<<Property>> ~maxZoom : float
-tileHighlightBatch : SpriteBatch = new SpriteBatch()
-buildingBatch : SpriteBatch = new SpriteBatch()
-tileHighlight : Texture = new Texture(Gdx.files.internal("map/tileHighlight.png"))
-errTileHighlight : Texture = new Texture(Gdx.files.internal("map/errTileHighlight.png"))
-isoTransform : Matrix4
-invIsoTransform : Matrix4
-canBuild : boolean
-selectedBuildingUpdated : boolean
+numberOfStudents : int
-minX : float = 0f
-maxX : float = 300f
-minY : float = -75f
-maxY : float = 75f
-buildingManager : BuildingManager
-mouseInWorld : Point
-btmLeft : Point
-topRight : Point
+selectedBuilding : Building
+satisfactionTracker : SatisfactionTracker = new SatisfactionTracker()
+moneyTracker : MoneyTracker = new MoneyTracker(1000)
+scoreTracker : ScoreTracker = new ScoreTracker()
+achievementManager : AchievementManager = new AchievementManager()
+achievementTracker : AchievementTracker = new AchievementTracker()
+achievementBar : AchievementBar
+World(achievementBar : AchievementBar)
+dispose() : void
+update() : void
+render() : void
+resize(width : int, height : int) : void
+clampCameraPosition() : void
+pan(x : float, y : float) : void
+panWithoutInertia(x : float, y : float) : void
+zoom(amount : float) : void
-updateZoom() : void
-updatePan() : void
+getZoom() : float
+getCursorGridPos() : Point
+highlightRegion(btmLeft : Point, topRight : Point, highlightTexture : Texture) : void
+getCameraPos() : Vector2
-gridPosToWorldPos(gridPos : Point) : Vector3
-initIsometricTransform() : void
+getMapTiles() : TiledMapTileLayer
+placeBuilding() : boolean
+removeBuilding(location : Point) : boolean
+getBuildingCount(type : BuildingType) : int
+reset() : void
+cursorOverBuilding() : boolean
+setStudentEnrolmentsIdenticalCount : int) : void
+getBuildings() : Iterable<Building>
+getBuildingCapacities(type : BuildingType) : Integer
-world
-world
-world
-world
-world
-managementMenu
ManagementMenu
-skin : Skin = new SkinCdx.files.internal("ui/uiskin.json")
-table : Table
-normalisedWidth : float = 0.35f
-normalisedHeight : float = 0.85f
-normalisedLeftPadding : float = (0.975f - normalisedWidth)
-normalisedTopPadding : float = (1 - normalisedHeight) / 2
-studentEnrollmentLabel : Label
-studentEnrollmentField : TextField
-background : ShapeActor
-world : World
-normalisedHiddenPadding : float = 1
-buildingCapacityTable : Table
-buildingCapacityLabels : Map<BuildingType, Label>
-buildingCapacityBars : Map<BuildingType, ProgressBar>
-resizableComponents : ResizableComponents
+ManagementMenu(stage : Stage, world : World)
-getTypeColor(type : BuildingType) : Color
-getBarStyle(type : BuildingType, height : int) : ProgressBarStyle
-updateCapacityTable() : void
-updateLabelPositions() : void
+updateElements() : void
-submitToWorld() : void
+resize(width : int, height : int) : void
+hide() : void
+show() : void
+toggleVisibility() : void
InfoBar
-infoTable : Table = new Table()
-titleTable : Table = new Table()
-skin : Skin = new SkinCdx.files.internal("ui/uiskin.json")
-satisfactionLabel : Label = new Label("", skin)
-titleLabel : Label = new Label("UniBum", skin)
-moneyLabel : Label
-timeLabel : Label
-scoreLabel : Label
-satisfactionModifierLabel : Label
-pauseTexture : Texture = new Texture("ui/pause.png")
-playTexture : Texture = new Texture("ui/play.png")
-pauseImage : Image = new Image(pauseTexture)
-playImage : Image = new Image(playTexture)
-timerLabelCell : Cell<Label>
-satisfactionLabelCell : Cell<Label>
-satisfactionModifierLabelCell : Cell<Label>
-scoreLabelCell : Cell<Label>
-pauseButtonCell : Cell<Image>
-moneyLabelCell : Cell<Label>
-bar : ShapeActor
-timer : Timer
-world : World
+InfoBar(stage : Stage, timer : Timer, world : World)
+update() : void
+resize(width : int, height : int) : void
+reset() : void
Timer
-remainingTime : float
<<Property>> ~initialTime : float
+hasFinished : boolean
+Timer(initialTime : float)
+tick(deltaTime : float) : boolean
+reset() : void
+getRemainingTime() : String
+isRunning() : boolean
~timer
~timer
-infoBar
-world
-world
BuildingPreviewMenu
-previewTable : Table
-background : Image
-normalisedWidth : float = 0.3f
-normalisedHeight : float = 0.25f
-normalisedLeftPadding : float = 0.01f
-normalisedTopPadding : float = 0.01f
-rightColumn : Table
-previewImage : Image
-previewCell : Cell<Image>
-nameLabel : Label
-sizeLabel : Label
-capacityLabel : Label
-costLabel : Label
-resizableComponents : ResizableComponents
-preview : Building
+BuildingPreviewMenu(stage : Stage)
+showContent(building : Building) : void
+hideContent() : void
+resize(width : int, height : int) : void
-previewMenu
BuildingMenu
-buildingImages : ArrayList<Image> = new ArrayList<>()
-navTableMap : HashMap<BuildingType, BuildingMenuEntry> = new HashMap<>()
-buildingInfoLabel : Label = new Label(
"", new SkinCdx.files.internal("ui/uiskin.json"))
)
-buildingInfoTable : Table = new Table()
-world : World
-bar : ShapeActor = new ShapeActor(GameState.UISecondaryColour)
-buildings : Building = new ArrayList<>()
-previewMenu : BuildingPreviewMenu
-currMenuEntry : BuildingMenuEntry
+BuildingMenu(stage : Stage, world : World)
+resize(width : int, height : int) : void
+update() : void
+reset() : void
-buildingMenu
-world


SatisfactionTracker
<<Property>> ~satisfaction : float
~modifiers : SatisfactionModifier
~satisfactionNeeds : SatisfactionNeeds
+SatisfactionTracker()
+getStringSatisfaction() : String
+updateSatisfaction(buildings : Iterable<Building>, previewBuilding : Building, totalStudents : int) : void
+getTotalModifierAdditions() : float
+getTotalModifierMultiplier() : float
+getLongestModifierRemainingDuration() : Duration
+addModifier(template : ScoreModifierTemplate, value : float, duration : Duration) : void
+addModifier(modifier : SatisfactionModifier) : void
+getScore() : float
+getUpdateInterval() : Duration
~satisfactionTracker
MoneyTracker
~MONEY_UPDATE_INTERVAL : Duration = Duration.ofSeconds(1)
~MONEY_UPDATE_AMOUNT_PER_STUDENT : int = 10
<<Property>> ~money : int
~lastUpdateTime : Instant
+MoneyTracker(initialMoney : int)
+subtractMoney(amount : int) : boolean
+updateMoney(studentCount : int) : void
+addMoney(amount : int) : void
~moneyTracker
EventBucket
-random : Random
-events : EventCard
+EventBucket()
-initializeEvents() : void
+getRandomEvent() : EventCard
~eventBucket
EventDisplay
-skin : Skin = new Skin(Gdx.files.internal("ui/uiskin.json"))
-dialog : Image
-table : Table
-eventTitleLabel : Label
-eventDescriptionLabel : Label
-choice1 : TextButton
-choice2 : TextButton
-choice3 : TextButton
-displayEndTime : Instant
-DISPLAY_TIME : Duration = Duration.ofSeconds(8)
~normalisedWidth : float = 0.4f
~normalisedHeight : float = 0.4f
~normalisedLeftPadding : float = (1 - normalisedWidth) / 2
~normalisedTopPadding : float = 0.5f
~normalisedHiddenPadding : float = 1
-dialogTexture : Texture
-timerBar : ShapeActor
-moneyTracker : MoneyTracker
-satisfactionTracker : SatisfactionTracker
-eventBucket : EventBucket = new EventBucket()
-eventCard : EventCard
-resizableComponents : ResizableComponents
-timerComponent : ResizableComponents
+EventDisplay(stage : Stage, moneyTracker : MoneyTracker, satisfactionTracker : SatisfactionTracker)
-hide() : void
-setEvent() : void
+show() : void
+update() : void
+resize(width : int, height : int) : void
-adjustTimerWidth() : void
~eventCard
EventCard
<<Property>> ~title : String
<<Property>> ~description : String
<<Property>> ~choices : ChoiceCard
+EventCard(title : String, description : String, choices : List<ChoiceCard>)

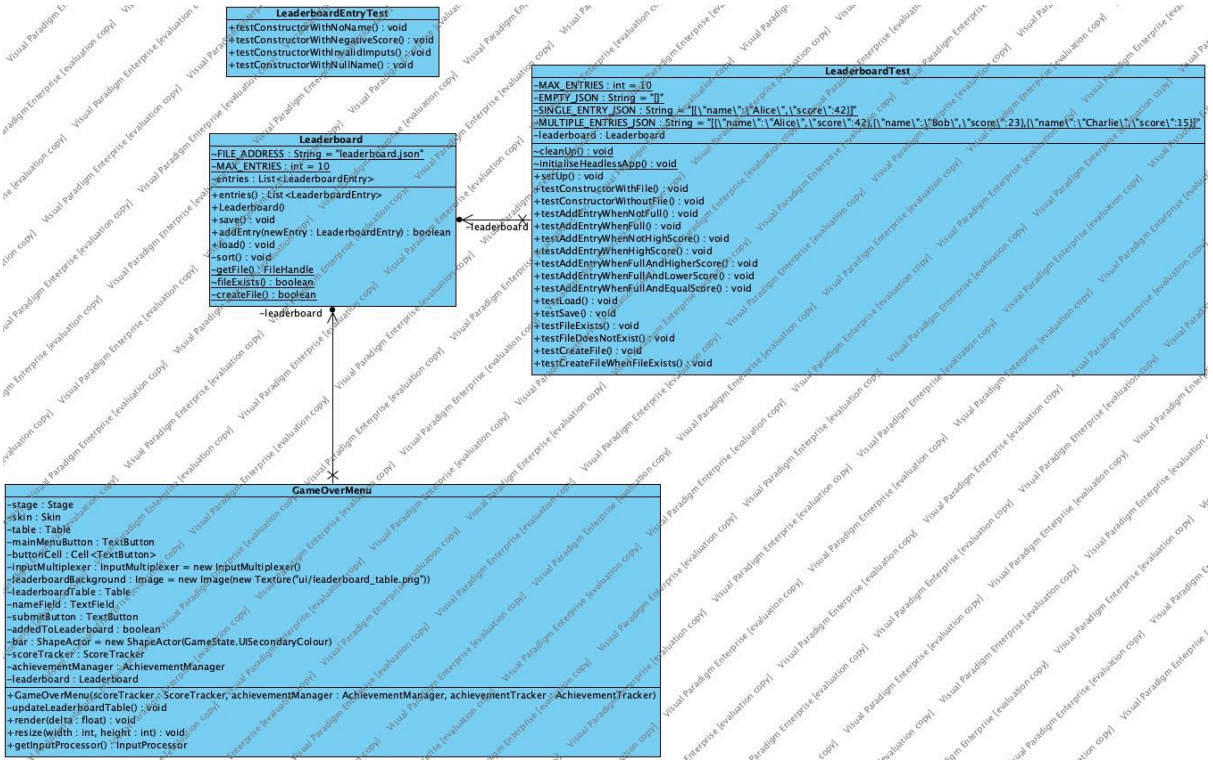

BuildingPreviewMenu
-previewTable : Table
-background : Image
-normalisedWidth : float = 0.3f
-normalisedHeight : float = 0.25f
-normalisedLeftPadding : float = 0.01f
-normalisedTopPadding : float = 0.01f
-rightColumn : Table
-previewImage : Image
-previewCell : Cell<Image>
-nameLabel : Label
-sizeLabel : Label
-capacityLabel : Label
-costLabel : Label
-resizableComponents : ResizableComponents
-preview : Building
+BuildingPreviewMenu(stage : Stage)
+showContent(building : Building) : void
+hideContent() : void
+resize(width : int, height : int) : void
-previewMenu
BuildingMenu
-buildingImages : ArrayList<Image> = new ArrayList<>()
-navTableMap : HashMap<BuildingType, BuildingMenuEntry> = new HashMap<>()
-buildingInfoLabel : Label = new Label(
"", new Skin(Gdx.files.internal("ui/uiskin.json"))
)
-buildingInfoTable : Table = new Table()
-world : World
-bar : ShapeActor = new ShapeActor(GameState.UISecondaryColour)
-buildings : Building = new ArrayList<>()
-previewMenu : BuildingPreviewMenu
-currMenuEntry : BuildingMenuEntry
+BuildingMenu(stage : Stage, world : World)
+resize(width : int, height : int) : void
+update() : void
+reset() : void
*
-buildings
-preview
Building
+texture : Texture
+textureScale : float
+textureOffset : Vector2
+flipped : boolean
+name : String
+capacity : int
+cost : int
+location : Point
+size : Point
+type : BuildingType
+Building(texture : Texture, textureScale : float, textureOffset : Vector2, location : Point, size : Point, flipped : Boolean, type : BuildingType, name : String, capacity : int, cost : int)
-currMenuEntry
BuildingMenuEntry
<<Property>> -navTable : Table
<<Property>> -buildingTable : Table
<<Property>> -costTable : Table
+BuildingMenuEntry()
+addToNavTable(actor : Actor) : void
+addToBuildingTable(actor : Actor) : void
+addToCostTable(actor : Actor) : void
+addToStage(stage : Stage) : void
+removeFromStage() : void
+resize(width : int, height : int) : void


ResizableComponents
<<Property>> ~stageWidth : float
<<Property>> ~stageHeight : float
~actors : List<Actor>
<<Property>> ~normalisedX : float
<<Property>> ~normalisedY : float
<<Property>> ~normalisedWidth : float
<<Property>> ~normalisedHeight : float
~ResizableComponents(width : float, height : float)
+addActor(actor : Actor) : void
+resize(width : int, height : int) : void
~positionComponents() : void
~positionComponent(actor : Actor) : void
+onScreen() : boolean
+addAction(actionGenerator : Function<Void, Action>) : void
+getX() : float
+getY() : float
+getWidth() : float
+getHeight() : float
~resizableComponents
~resizableComponents
ManagementMenu
~skin : Skin = new Skin(Gdx.files.internal("ui/uiskin.json"))
~table : Table
~normalisedWidth : float = 0.35f
~normalisedHeight : float = 0.85f
~normalisedLeftPadding : float = (0.975f - normalisedWidth)
~normalisedTopPadding : float = (1 - normalisedHeight) / 2
~studentEnrollmentLabel : Label
~studentEnrollmentField : TextField
~background : ShapeActor
~world : World
~normalisedHiddenPadding : float = 1
~buildingCapacityTable : Table
~buildingCapacityLabels : Map<BuildingType, Label>
~buildingCapacityBars : Map<BuildingType, ProgressBar>
~resizableComponents : ResizableComponents
+ManagementMenu(stage : Stage, world : World)
~getTypeColor(type : BuildingType) : Color
~getBarStyle(type : BuildingType, height : int) : ProgressBarStyle
~updateCapacityTable() : void
~updateLabelPositions() : void
+updateElements() : void
~submitToWorld() : void
+resize(width : int, height : int) : void
+hide() : void
+show() : void
+toggleVisibility() : void
~managementMenu
WorldInputProcessor
~cursorPos : int[] = new int[2]
~cursorPosWhenClicked : int[] = new int[2]
~clickedOnWorld : boolean = false
~draggedSinceClick : boolean = true
~moveUp : boolean = false
~moveDown : boolean = false
~moveLeft : boolean = false
~moveRight : boolean = false
~zoomIn : boolean = false
~zoomOut : boolean = false
~world : World
+achievementTracker : AchievementTracker
~managementMenu : ManagementMenu
+WorldInputProcessor(world : World, managementMenu : ManagementMenu)
+keyDown(keycode : int) : boolean
+keyUp(keycode : int) : boolean
+keyTyped(character : char) : boolean
+touchDown(x : int, y : int, pointer : int, button : int) : boolean
+touchUp(x : int, y : int, pointer : int, button : int) : boolean
+touchDragged(x : int, y : int, pointer : int) : boolean
+touchCancelled(screenX : int, screenY : int, pointer : int, button : int) : boolean
+mouseMoved(x : int, y : int) : boolean
+scrolled(amountX : float, amountY : float) : boolean
+update(dt : float) : void
BuildingPreviewMenu
~previewTable : Table
~background : Image
~normalisedWidth : float = 0.3f
~normalisedHeight : float = 0.25f
~normalisedLeftPadding : float = 0.01f
~normalisedTopPadding : float = 0.01f
~rightColumn : Table
~previewImage : Image
~previewCell : Cell<Image>
~nameLabel : Label
~sizeLabel : Label
~capacityLabel : Label
~costLabel : Label
~resizableComponents : ResizableComponents
~preview : Building
+BuildingPreviewMenu(stage : Stage)
+showContent(building : Building) : void
+hideContent() : void
+resize(width : int, height : int) : void
~preview
Building
+texture : Texture
+textureScale : float
+textureOffset : Vector2
+flipped : boolean
+name : String
+capacity : int
+cost : int
+location : Point
+size : Point
+type : BuildingType
+Building(texture : Texture, textureScale : float, textureOffset : Vector2, location : Point, size : Point, flipped : Boolean, type : BuildingType, name : String, capacity : int, cost : int)

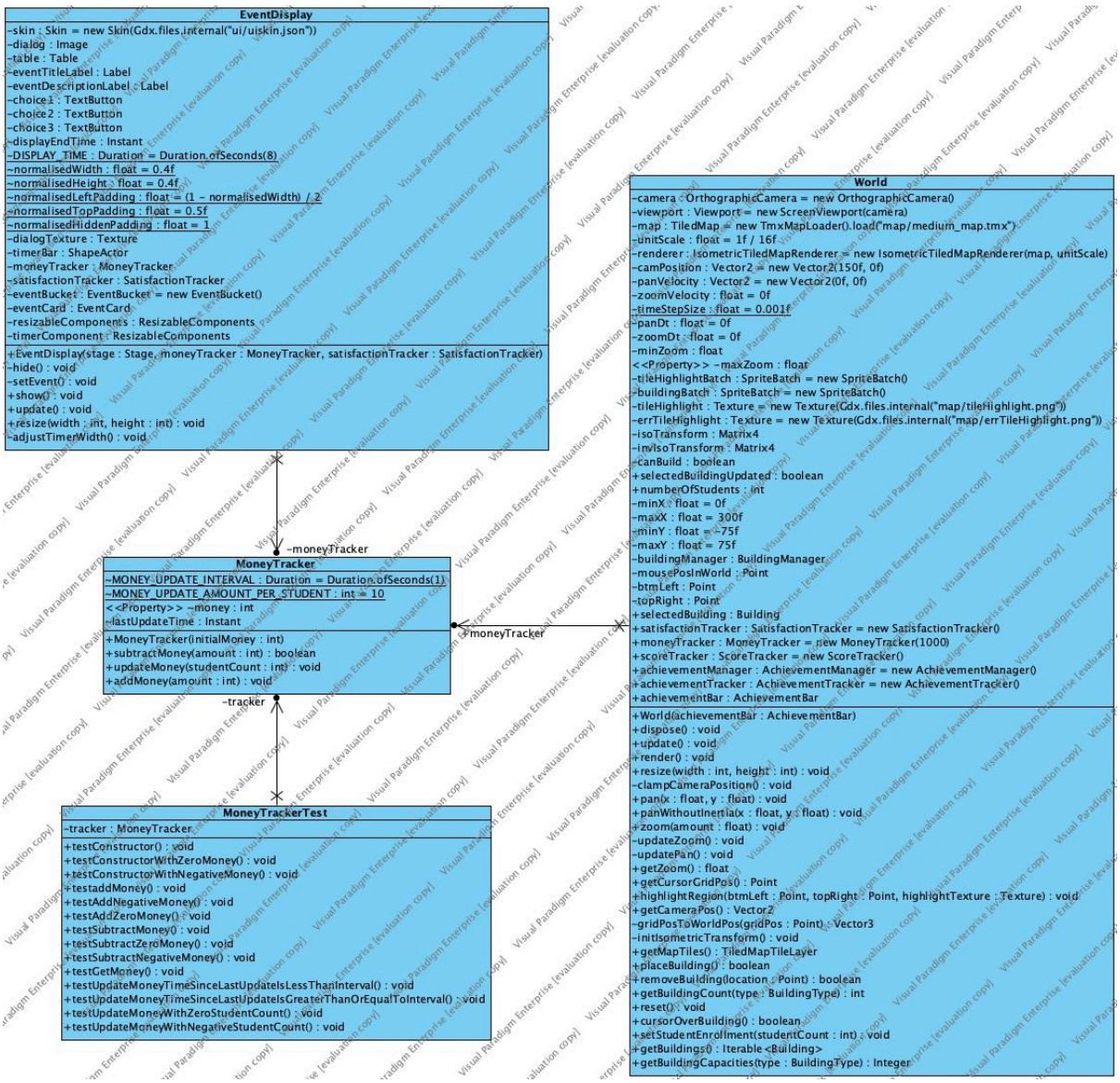
Scoring: The scoring package contains classes that modify the student satisfaction as well as the player's score. These classes are used by many others as shown in the diagram below.



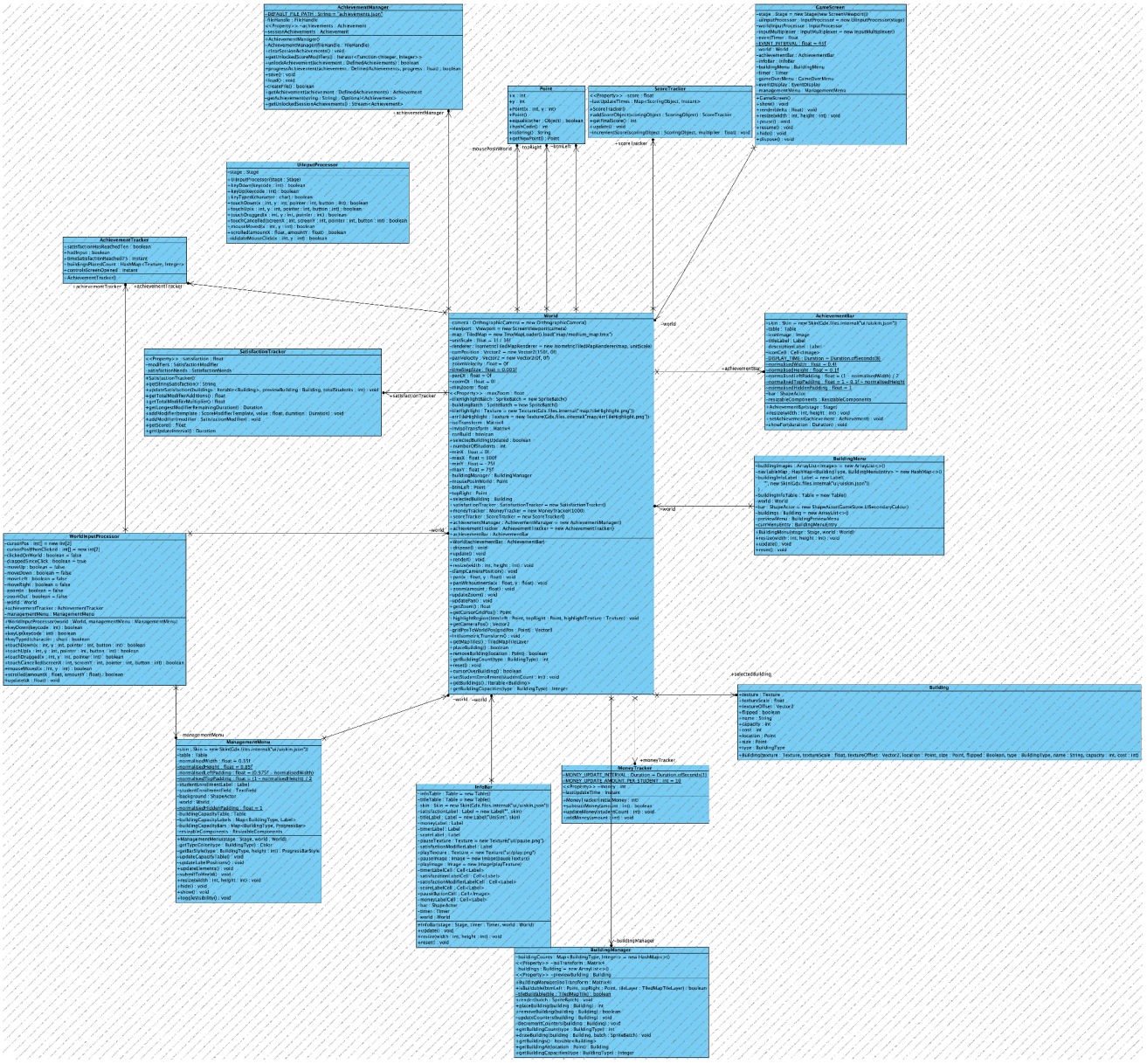
Leaderboard: The leaderboard package contains just two classes that allow for the leaderboard requirement to be satisfied.



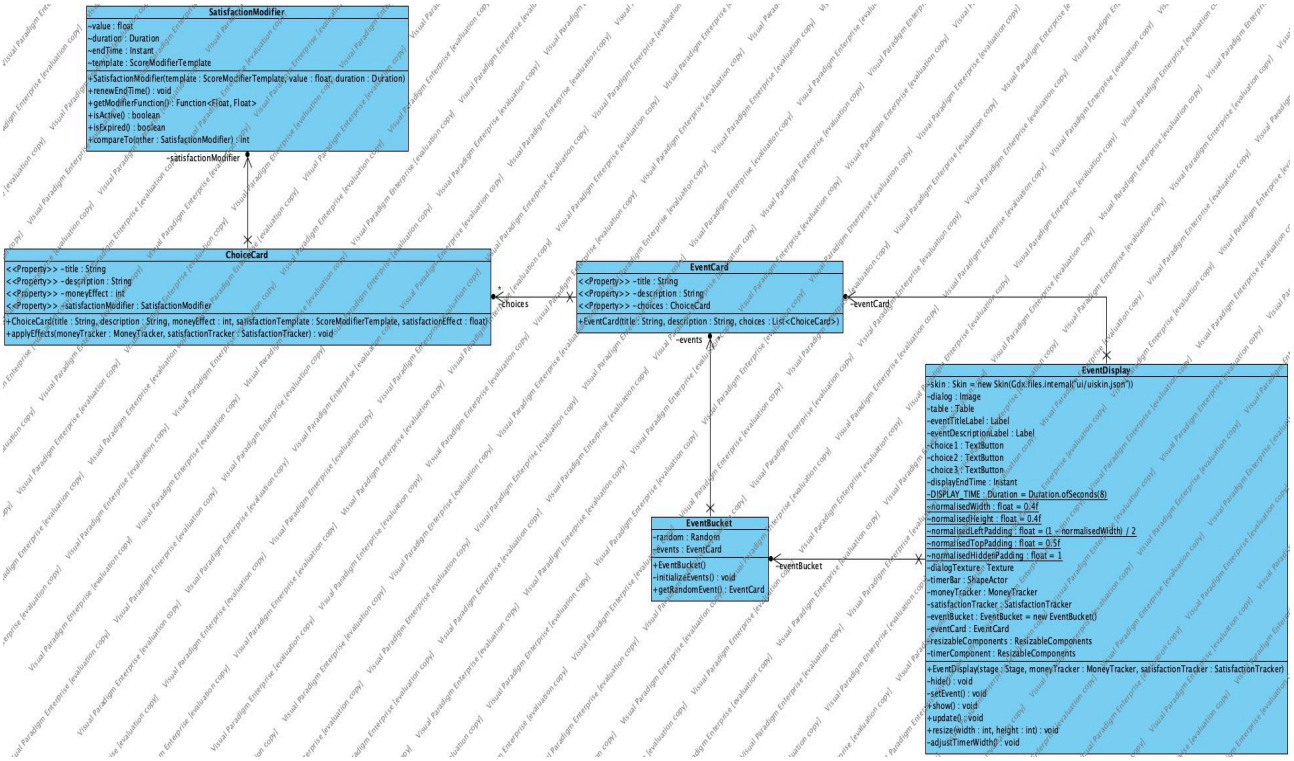
Finance: The finance package controls the money awarded to the player and tracks how much money the player has. The MoneyTracker class only has 4 attributes and 4 public methods which are only used by the World class.



World: There is a full size image of this diagram available on the website if it is too small to read the individual words here. The World package contains three classes, UIInputProcessor, World and WorldInputPorcessor. The World class holds all of the gameplay elements such as the achievements, controls the rendering of the game and updates the state based on the user's choices. WorldInputProcessor is the class responsible for controlling the game based on the user's input. The diagram below shows which classes use the methods from classes in the World package.

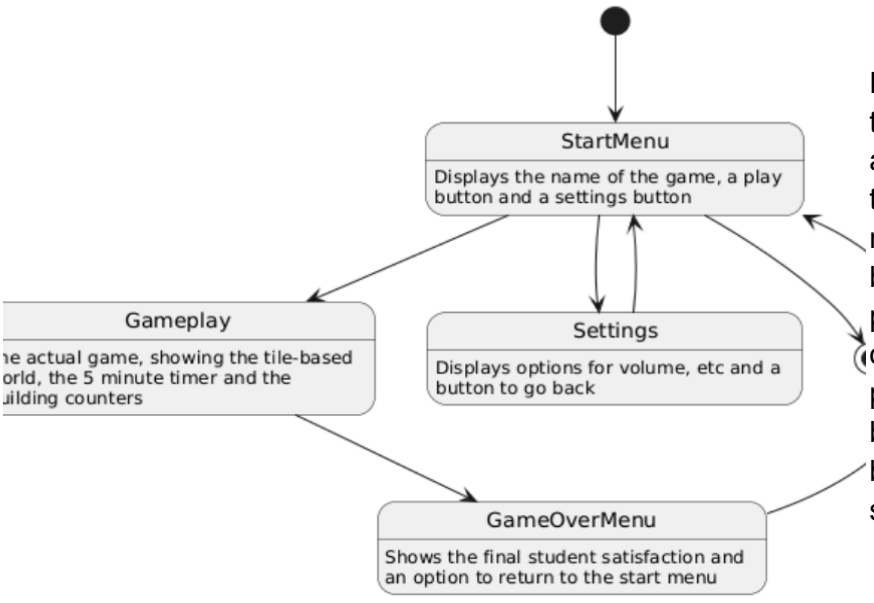


Events: The events package contains three classes which are used to randomly select events, and provide a way for the user to pick their response to each event.



Behavioural Diagrams

State Diagram



Here we describe how the game starts paused and then once unpaused, the user can navigate the map, change the type of building being placed, place a building, and, once a building has been placed, can remove a building. The game can be paused from any state.

Sequence Diagram

This diagram outlines a typical user interaction with the game.